Smart Attendance & Lecture Management System Documentation

Overview

The Smart Attendance and Lecture Management System is a mobile-based solution designed to help university students and professors (doctors) manage their weekly lectures and attendance seamlessly. The system automates the process of taking attendance using computer vision by identifying students through facial recognition and synchronizing the data with the university's lecture schedule.

The project provides two types of accounts:

- **Student Account**: Allows students to register, view their weekly schedule, and track attendance history.
- **Doctor Account**: Allows professors to register, manage their courses, and automatically receive attendance reports after each lecture.

The main innovation lies in the AI-based face recognition module, which automatically detects students' faces during lectures and sends accurate attendance data to the database without manual input.

This prototype version runs the backend locally and uses ngrok to expose it publicly over HTTPS, allowing the Flutter mobile app on physical devices to connect seamlessly to the local server for end-to-end testing with a small dataset (10–20 students, 2–3 courses).

Core Features

Student Features

- Register using university email and personal details.
- Capture 5 facial images (different poses and lighting) to create embeddings for recognition.
- View personal timetable and attendance record.
- Secure login and profile management.

Doctor Features

- Register with name, courses taught, and login credentials.
- Automatically receive attendance reports for each lecture.
- View attendance statistics and student participation.

AI Module

- Captures live images from the camera in each lecture hall.
- Detects and recognizes student faces using a pre-trained face recognition model.
- Generates embeddings for new faces and compares them to existing ones in the PostgreSQL database using the pgvector extension.
- Sends attendance data to the backend automatically after each lecture.

System Architecture

The system is built with a modular architecture to ensure scalability and ease of integration:

Component	Description	Technology
Mobile App	Interface for students and professors to manage accounts and schedules.	Flutter
Backend API	Handles authentication, data processing, AI orchestration, and database operations.	FastAPI (Python)
Database	Stores user data, embeddings, courses, and attendance records.	PostgreSQL (with pgvector prepared for future use)
AI Module	Face detection, embedding generation, and matching.	InsightFace (buffalo_sc model), OpenCV, NumPy
ORM	Async database interactions.	SQLModel
Authentication	JWT-based secure access to protected endpoints.	JWT with bcrypt hashing

Component	Description	Technology
Local Testing	Backend exposed via ngrok for mobile connectivity.	Uvicorn + pyngrok

Data Flow

1. **User Registration** Mobile App → POST /users (JSON with email, password, full name, role) → FastAPI → Hashes password → Inserts new user into PostgreSQL → Returns success.
2. **Face Enrollment (Student)** Mobile App → Captures 5 photos → Converts to base64 strings → POST /enroll (user_id + list of base64 images) → FastAPI → Decodes base64 using OpenCV → InsightFace generates embeddings → Embeddings converted to JSON strings → Stored in embeddings table linked to user_id.
3. **Login** Mobile App → POST /token (email/username + password) → FastAPI → Verifies hashed password in database → Issues JWT access token → Token stored in app and sent as Bearer header in future requests.
4. **Course & Lecture Management (Doctor)** Mobile App → POST /courses (course details) and POST /lectures (schedule) → FastAPI (protected by JWT) → Inserts into PostgreSQL.
5. **Taking Attendance (Doctor)** Mobile App → Captures lecture photo → Converts to base64 → POST /attendance/image (course_id, lecture_date, base64 image(s), threshold) → FastAPI → Decodes image → InsightFace detects faces and generates query embeddings → Compares each against all stored embeddings using cosine similarity → Identifies matching students (above threshold) → Inserts attendance records (present/absent) into database → Returns list of recognized students with confidence scores.
6. **Viewing Attendance** Mobile App → GET endpoints (e.g., /courses/{id}/attendance) with JWT → FastAPI → Queries database → Returns attendance data to display in app.

All image transfers use base64 encoding to avoid multipart complexity in early prototype.

Team Roles and Responsibilities

- **Mohamed — Computer Vision Engineer**: Led facial recognition implementation and model integration.
- **Mina Waheed — Computer Vision & Backend Engineer**: Worked on AI processing pipeline, embedding handling, and backend endpoints related to face recognition; also handled camera integration and data collection.
- **Mark — Mobile App Developer**: Developed core Flutter screens and API integration.
- **Ahmed Yusry — Mobile App Developer**: Focused on UI/UX, camera access, and image capture flows; developed core screens and image capture flows.
- **Mina Naseeh — Backend Engineer**: Designed database schema and core API logic.
- **Ayad — Backend Engineer**: Handled authentication, integration between components, and deployment setup.

Development Process

We followed an **Agile approach** throughout the project, working in short iterations with frequent feedback and adjustments rather than a fixed plan. This allowed us to respond quickly to issues that arose during integration.

The original timeline outlined weekly goals (research, tool learning, database design, backend, mobile app, AI integration), but in practice, **we didn’t follow it in detail**. Tasks overlapped, priorities shifted, and some phases took longer than expected.

The majority of development time was spent on the **backend and integrating all components** — connecting Flutter to FastAPI, handling base64 image flow reliably, debugging AI model loading, ensuring async database operations worked smoothly, and fixing JWT authentication across endpoints. These integration challenges required multiple iterations and consumed far more effort than initially anticipated.

For mobile testing, we extensively used **ngrok** through the provided share_api.py script. Running python share_api.py starts the local FastAPI server and automatically creates a temporary public HTTPS tunnel (e.g., https://xxxx.ngrok-free.dev). This URL is then set as the base API URL in the Flutter app, enabling real-device testing without needing both devices on the same local network.

Challenges Encountered

- Integration complexity between Flutter, FastAPI, InsightFace, and PostgreSQL.
- Ensuring consistent base64 decoding and AI processing across environments.
- Frequent ngrok URL changes requiring updates in the Flutter app during testing sessions.
- Balancing AI accuracy with prototype performance constraints.